TuneSwipe Project Documentation

https://ec-327-project.vercel.app/

1. Project Overview and Motivation

TuneSwipe is a web application designed to make music discovery feel fast, visual, and interactive. Instead of browsing a long list of songs or manually building a playlist, the user is shown one song card at a time and can either like or pass the track. The design is intentionally similar to a swipe-based recommendation interface: each card contains the album image, song title, artist name, album name, popularity indicator, and a play button for an audio preview when a preview is available. The main goal of the project is to reduce the amount of effort required to discover music while still giving the user control over which songs are saved.

The project combines several important full-stack concepts. The frontend is built in React with Vite, the backend is a Node.js and Express server, user accounts are handled with Firebase Authentication, user data is stored in Cloud Firestore, and song metadata is requested from Spotify. This makes the project more than a static interface: it has real authentication, persistent user-specific data, backend API routes, third-party API communication, and deployment considerations.

The motivation for building the project came from the idea of making music recommendations feel more personal and engaging. A normal playlist app often requires the user to search for songs, browse large lists, or already know what they want. TuneSwipe reverses that process by bringing songs to the user and allowing the recommendation pool to evolve based on the user's liked songs. If a user likes a song by a particular artist, the app can use that artist as a future search influence, making the experience feel less random over time.

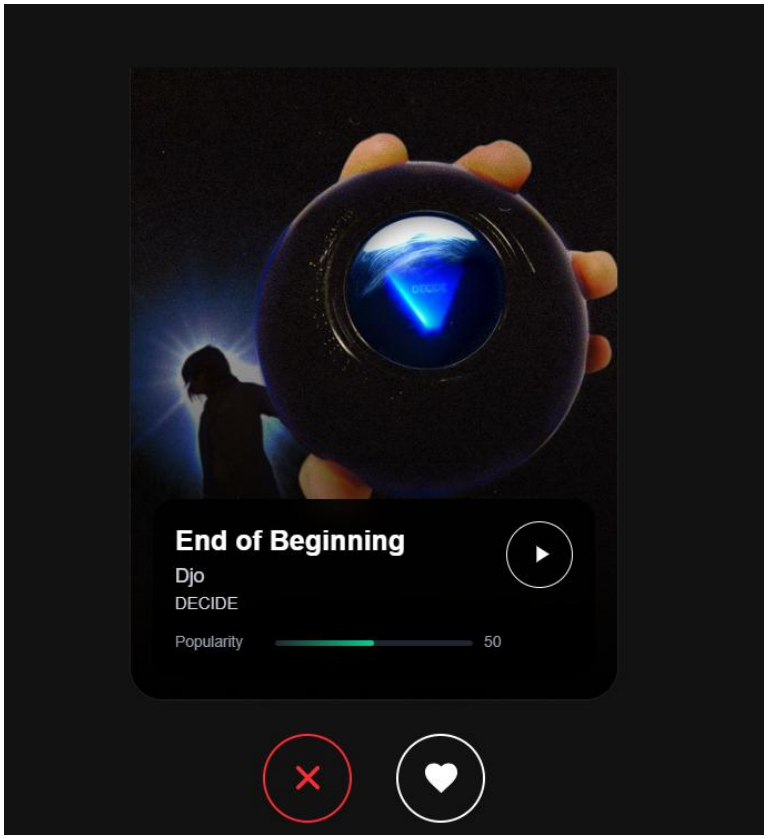

Figure 1: Song Card

2. Goals, Scope, and Final Feature Set

The final scope of TuneSwipe was to create a functional music discovery application rather than only a visual prototype. The system needed to load real song information, allow the user to interact with songs, save those interactions, and keep each user's activity separate. The feature set was developed in stages so the team could first make the basic card interface work, then connect it to the backend, then add persistence, deduplication, authentication, and deployment.

3. How the Application Works

When the application opens, the user is first directed to the authentication screen. This is important because the recommendation and history logic depend on knowing which user is active. Once the user signs in or creates an account, Firebase returns a user object with a unique user ID. That UID becomes the key used by both the frontend and backend to separate one user's liked and passed songs from another user's data.

After login, the frontend requests songs from the backend using a route similar to /songs?q=pop&uid=<userId>. The query controls what kind of tracks are requested, while the UID tells the backend which Firestore user history should be used for filtering. The backend then communicates with Spotify to retrieve candidate tracks, maps the Spotify response into a simpler format, removes songs that the current user has already liked or passed, and returns the final list to the frontend.

The frontend stores the returned songs in a deck array. The top song in the deck is rendered as the active card. The next two songs can be rendered behind it to create the feeling of a stack. When the user presses like or pass, the current track is removed from the local deck immediately so the interface

feels responsive. At the same time, the frontend sends a POST request to the backend so the track is saved in Firestore under either likedSongs or passedSongs for the current user.

If the deck becomes small, the app can request more songs. For users with no saved history, a generic query such as “pop” is used. For users with liked songs, the app can use the artist or title of a recently liked track as the next query. This was a practical recommendation strategy for the project because it is simple enough to implement within the project timeline, but still makes the recommendations reflect the user’s behavior.

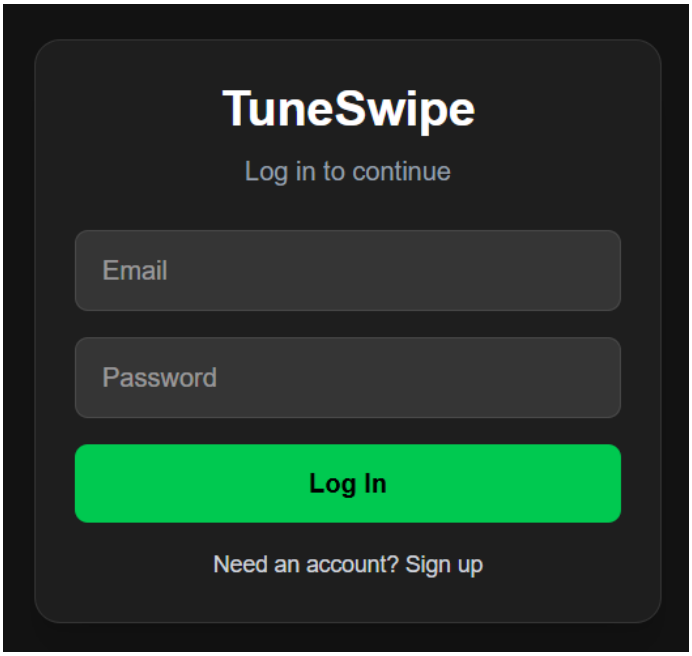

Figure 2: Login/Signup Page

4. System Architecture

TuneSwipe follows a client-server architecture. The React frontend is responsible for presentation, user interaction, authentication state, and calling backend routes. The Express backend is responsible for securely communicating with Spotify, saving song actions to Firestore, filtering out songs that should not be shown again, and returning normalized data to the frontend. Firestore acts as the persistent database, while Firebase Authentication manages account creation and login.

A major design decision was to keep Spotify credentials only on the backend. The Spotify client secret should never be placed in the React frontend because frontend environment variables are included in the browser build. Keeping Spotify API calls on the Express server allows the backend to request access tokens securely and send only the necessary song data back to the browser.

Vite is used as the frontend development and build tool. During development, Vite runs the React app locally and provides fast refresh. In production, the frontend can be deployed to Vercel as a static React build. The backend is not hosted by Vite; it is hosted separately on Render as a Node web service. This separation is important because the frontend and backend have different responsibilities and deployment requirements.

Layer	Technology	Responsibility
Frontend	React + Vite	Renders the interface, handles state, calls backend routes, and manages Firebase auth state.
UI Motion	Framer Motion	Controls card animation, swipe behavior, hover effects, and button interactions.
Backend	Node.js + Express	Provides API routes for songs, likes, passes, previews, and user-specific data.
Database	Cloud Firestore	Stores liked and passed songs under each user document.
Authentication	Firebase Authentication	Creates accounts and provides the UID used to isolate user data.
External API	Spotify Web API / preview URLs	Provides track metadata and preview links when available.
Hosting	Vercel + Render	Vercel hosts the frontend; Render hosts the backend service.

5. Main Data Flow

The most important flow in the project is the song discovery flow. It connects authentication, backend routing, Spotify, Firestore filtering, and React rendering. The user logs in through Firebase Authentication, the frontend receives the current user UID, the frontend chooses an initial query, and the frontend calls the backend /songs route with both the query and UID. The backend requests Spotify track results, converts them into the project's song object format, reads the current user's likedSongs and passedSongs collections, filters out already-seen IDs, and returns the filtered list to React.

This flow keeps the frontend simple and protects backend logic that depends on Spotify credentials and Firestore queries. The frontend does not need to know how Spotify tokens are created or how filtering is implemented; it only receives an array of tracks ready to display.

6. Algorithms and Application Logic

The frontend represents song recommendations as a deck. The first item in the array is the active card, and the next items are upcoming cards. When a track is liked or passed, the app removes that track from the deck using its Spotify ID. This prevents the same card from staying visible after an action and makes the next recommendation appear immediately. The deck also avoids adding duplicate tracks by comparing new track IDs against IDs that are already present in state.

The backend uses Firestore as the source of truth for which tracks should not be recommended again. Liked and passed songs are stored in separate subcollections under the same user. Using the Spotify track ID as the document ID makes lookup and deduplication much easier because each track has a stable identifier. Before returning results to the frontend, the backend builds a set of already-seen IDs and removes those songs from the Spotify results.

TuneSwipe does not use a machine learning recommendation model. Instead, it uses a lightweight query-based strategy. For a new user, the app starts with a broad genre or popular query. Once the user has liked songs, the app can use the artist or title from a recent liked song as the next search term. This creates a simple feedback loop: the user's likes influence future candidate tracks.

Audio previews are handled carefully because not every Spotify track result includes a usable preview URL. Some tracks return an empty preview field, and in those cases the app should not promise that audio is available. The final implementation attempts to use the preview URL included with the track first. If that value is missing, the app can request a preview fallback from the backend. If a preview URL is eventually available, the HTML audio element plays it when the user clicks the play button. If no preview can be resolved, the interface displays a graceful "No preview available" message rather than breaking the card.

```
setDeck((prev) => {
  const existingIds = new Set(prev.map((track) => track.id));
  const newTracks = formatted.filter((track) => !existingIds.has(track.id));
  return append ? [...prev, ...newTracks] : newTracks;
});

const seenIds = new Set([...likedIds, ...passedIds]);
const filteredSongs = songs.filter((song) => !seenIds.has(song.spotifyId));
res.json(filteredSongs);

if (!Array.isArray(likedSongs) || likedSongs.length === 0) {
  return 'pop';
}
const recent = likedSongs[likedSongs.length - 1];
return recent.artist || recent.title || 'pop';
```

7. Key Code Snippets and Technical Implementation

The project's key code is split between the React components and the Express backend routes. React handles the user experience, while Express handles data retrieval and persistence. The following snippets show the most important implementation patterns without listing the entire codebase.

When a user likes a song, the frontend sends the track object and UID to the backend. The backend validates that the request contains a UID and Spotify ID, then saves the song under users/{uid}/likedSongs/{spotifyId}. This structure keeps each user's liked songs separate and prevents duplicate documents for the same song.

```
app.post('/like', async (req, res) => {
  const { uid, song } = req.body;
  if (!uid || !song?.spotifyId) {
    return res.status(400).json({ error: 'Missing uid or song data' });
  }
  await db.collection('users').doc(uid)
    .collection('likedSongs').doc(song.spotifyId)
    .set({ ...song, likedAt: new Date() });
  res.json({ message: 'Song saved successfully' });
});
```

```
useEffect(() => {
  const unsubscribe = onAuthStateChanged(auth, (currentUser) => {
    setUser(currentUser);
    setAuthLoading(false);
  });
  return () => unsubscribe();
}, []);
```

8. Design Tradeoffs and Challenges

Several tradeoffs were made during development. The first major tradeoff was using a query-based recommendation approach instead of a complex recommendation algorithm. This was appropriate for the course project because it allowed the team to build a working full-stack system in the available time, while still demonstrating personalization through liked-song history. A more advanced version could use Spotify audio features, collaborative filtering, or a larger recommendation model, but those approaches would require more time, more data, and additional privacy considerations.

A second tradeoff involved audio previews. The application can only play audio when a preview URL is available. Some Spotify track results do not provide previews, and this limitation is outside the control of the frontend. The project handles this by attempting a fallback preview lookup and by clearly showing the user when no preview can be played. This is more honest and reliable than showing a play button that silently fails.

A third challenge was keeping the user interface visually balanced. The song card, information panel, play button, and like/pass buttons all needed to fit on different screen sizes. Small changes to margins and container height could cause the buttons to overlap the card or fall too low on the screen. The final layout uses a centered container, a controlled card height, and spacing between the card and action buttons so the interface remains usable.

The final important tradeoff was storing liked and passed songs in Firestore rather than only in local state. Local state would have been simpler, but it would reset when the page refreshed and would not work for multiple users. Firestore made the project more complex but allowed persistent data, account-specific history, accurate counts, and filtering across sessions.

9. Testing and Debugging Process

Testing was done incrementally because the project had several moving parts. The backend was tested first using direct browser requests and curl commands to confirm that the routes returned JSON and that Firestore writes were successful. For example, the /like and /pass routes were tested by sending sample song objects and then checking Firestore to verify that the documents were created under the correct user.

The frontend was tested using the browser console and React development output. Console logs were used to confirm when songs were fetched, what arrays were returned from the backend, whether preview URLs existed, and whether like/pass handlers were being called. Several bugs were found this way, including CORS errors, empty arrays from the backend, invalid Spotify limit values, missing preview URLs, and authentication state issues.

One important debugging lesson was that a 500 error in the frontend is usually not a React problem by itself. It often means the backend route crashed or returned an error. For example, an “Invalid limit” message came from the Spotify API because the backend requested a number of tracks outside

Spotify's accepted range. Fixing that required changing the backend route, not the frontend card rendering code.

10. Deployment and Hosting Plan

The project is structured so the frontend and backend can be deployed separately. The React/Vite frontend is hosted on Vercel because Vercel can build the static React app and serve it quickly. The backend is hosted on Render because Render can run a persistent Node.js Express server. This separation also matches the architecture of the project: the browser loads the interface from Vercel, while API requests go to the Express server on Render.

For the frontend deployment, Vercel should be connected to the GitHub repository. The build command is normally npm run build, and the output directory is dist for a Vite project. Any frontend environment variable that stores the backend URL should use Vite's VITE_ prefix, such as VITE_API_URL. The frontend should not contain the Spotify client secret or the Firebase service account key.

For the backend deployment, Render should be configured as a Node web service pointing to the backend folder. The start command should run the Express server, commonly npm start or node server.js depending on the package.json scripts. Backend environment variables should include the Spotify client ID, Spotify client secret, and Firebase Admin credentials or a secure method of loading the service account. The backend must also allow requests from the deployed Vercel frontend through CORS.

In production, the frontend fetch calls should not point to localhost. During development, localhost:3000 works because the backend is running on the same computer. After deployment, the frontend must call the Render backend URL. This can be handled by using an API_URL constant that reads from an environment variable, allowing the same codebase to work locally and online.

```
const API_URL = import.meta.env.VITE_API_URL || 'http://localhost:3000';
fetch(`${API_URL}/songs?q=${encodeURIComponent(query)}&uid=${user.uid}`);
```

11. Future Improvements

A future version of TuneSwipe could add stronger integration with a user's Spotify account. The current project uses Spotify for track metadata and preview information, but liking a song inside TuneSwipe is stored in Firestore. With Spotify OAuth, the app could request permission from the user and then save liked songs directly to the user's Spotify library or create a playlist. This would require a different authentication flow from the current client-credentials approach because actions on a user's Spotify account require user authorization.

The recommendation system could also become more advanced. Instead of using only a recent liked artist or title, the app could combine multiple liked artists, track popularity, genres, and audio features. Another improvement would be to add playlist creation, search controls, better handling for unavailable audio previews, and profile pages showing the user's liked-song history.

Overall, the completed project demonstrates a working full-stack application with authentication, persistent user data, Spotify-based metadata, interactive UI design, backend filtering, and deployment planning. The final version is not just a mockup; it contains the core systems needed for a real music discovery app.